

Módulo 6: Métodos Estatísticos

e Mineração Não-Supervisionada

Universidade do Estado do Pará (UEPA) – CCNT
Curso: Engenharia de Software, 8º Semestre
Disciplina: Mineração de Dados | Carga Horária: 80h

1 Objetivos de Aprendizagem

- ✓ **Aplicar** testes de hipótese estatísticos para comparar o desempenho de algoritmos de mineração de dados, interpretando p-valores e erros Tipo I e II.
- ✓ **Calcular** e interpretar medidas de correlação e dependência (Pearson, Spearman, Informação Mútua) para análise exploratória de dados.
- ✓ **Implementar** e comparar algoritmos de agrupamento (K-Means, Hierarchical, DBSCAN, GMM) para descoberta de estrutura em dados não rotulados.
- ✓ **Selecionar** o algoritmo de clustering mais adequado conforme a forma geométrica dos grupos, presença de ruído e dimensionalidade dos dados.
- ✓ **Avaliar** agrupamentos com métricas internas (Silhouette, Davies-Bouldin) e externas (ARI, NMI), compreendendo as limitações de cada abordagem.
- ✓ **Minerar** regras de associação usando os algoritmos Apriori e FP-Growth, calculando suporte, confiança, lift e conviction.
- ✓ **Detectar** anomalias usando métodos estatísticos, baseados em distância e em florestas de isolamento (Isolation Forest).
- ✓ **Identificar** as tendências do estado da arte (2020–2024) em clustering profundo, HDBSCAN e detecção moderna de outliers.

2 Fundamentos Estatísticos para Mineração de Dados

2.1 Testes de Hipótese

Um **teste de hipótese** é um procedimento para decidir, com base em evidências amostrais, se uma afirmação sobre uma população deve ser aceita ou rejeitada.

Componentes de um Teste de Hipótese

- H_0 (**hipótese nula**): A posição padrão a ser testada (ex. “algoritmo A e B têm a mesma acurácia”).
- H_1 (**hipótese alternativa**): O que queremos provar (ex. “algoritmo A é melhor que B”).

- **Nível de significância α :** Probabilidade máxima do Erro Tipo I aceitável (tipicamente $\alpha = 0.05$).
- **p-valor:** Probabilidade de observar um resultado tão extremo quanto o observado, supondo H_0 verdadeira. Rejeita-se H_0 se $p < \alpha$.

Tipos de Erro:

- **Erro Tipo I (α):** Rejeitar H_0 quando ela é verdadeira (*falso positivo*).
- **Erro Tipo II (β):** Não rejeitar H_0 quando ela é falsa (*falso negativo*).
- **Poder do Teste ($1 - \beta$):** Probabilidade de detectar corretamente o efeito quando H_1 é verdadeira.

Testes Relevantes para Mineração de Dados:

- **Teste t pareado:** Compara médias de dois classificadores nos mesmos folds de validação cruzada. Assume normalidade das diferenças.

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad \bar{d} = \frac{1}{n} \sum_{i=1}^n (a_i - b_i)$$

- **ANOVA (Analysis of Variance):** Compara três ou mais classificadores simultaneamente. Decompõe variância total em variância entre grupos e dentro dos grupos.
- **Teste de Wilcoxon Signed-Rank** (não-paramétrico): Alternativa ao teste t pareado quando os dados não seguem distribuição normal. Ranqueia as diferenças absolutas e testa se as medianas são iguais.
- **McNemar's Test:** Compara dois classificadores pelas matrizes de confusão. Testa se o número de casos onde A acerta e B erra é diferente do número onde B acerta e A erra:

$$\chi^2 = \frac{(b - c)^2}{b + c}$$

onde b = casos em que A acerta e B erra, c = casos em que B acerta e A erra.

Comparações Múltiplas

Ao comparar k classificadores com múltiplos testes de hipótese, a probabilidade de pelo menos um falso positivo é $1 - (1 - \alpha)^k$. Para 10 comparações com $\alpha = 0.05$, isso é $\approx 40\%$! Use correções como **Bonferroni** ($\alpha' = \alpha/k$) ou o teste de **Friedman** + post-hoc de Nemenyi para comparações múltiplas.

2.2 Correlação e Dependência

Coefficiente de Correlação de Pearson (r): Mede relação linear entre duas variáveis contínuas:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}} \in [-1, 1]$$

$r = 1$ (correlação positiva perfeita), $r = 0$ (sem correlação linear), $r = -1$ (negativa perfeita).

Correlação de Spearman (ρ): Aplica Pearson nos **postos** (*ranks*) dos dados. Detecta monotonicidade, não apenas linearidade. Robusta a outliers e útil para dados ordinais.

Tau de Kendall (τ): Baseado em concordância/discordância de pares. Mais robusto que Spearman para amostras pequenas.

Informação Mútua $I(X; Y)$: Mede dependência geral (não apenas linear) entre variáveis:

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

$I(X; Y) = 0$ iff X e Y são independentes. Muito usado em seleção de features (MRMR).

2.3 Distribuições Relevantes

Tabela 1: Distribuições de probabilidade frequentes em Mineração de Dados.

Distribuição	FDP/FMP	Suporte	Uso em DM
Normal	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	$\mathbb{R}$	Ruído, GMM, testes paramétricos
Bernoulli	$p^x(1-p)^{1-x}$	$\{0, 1\}$	Classificação binária, Naive Bayes
Binomial	$\binom{n}{k} p^k (1-p)^{n-k}$	$\{0, \dots, n\}$	Contagem de acertos, limiar de confiança
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	$\mathbb{N}_0$	Contagem de eventos raros, detecção de anomalias
Exponencial	$\lambda e^{-\lambda x}$	$[0, +\infty)$	Tempo entre eventos, detecção de outliers
t -Student	$\frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	$\mathbb{R}$	Testes com pequenas amostras
Qui-Quadrado	$\frac{x^{k/2-1} e^{-x/2}}{2^{k/2}\Gamma(k/2)}$	$[0, +\infty)$	Teste de independência (tabelas de contingência)

3 Agrupamento (Clustering)

3.1 Definição e Aplicações

O **agrupamento** (clustering) é uma tarefa de aprendizado não-supervisionado que busca descobrir a estrutura natural dos dados, organizando exemplos similares em grupos (*clusters*) sem o uso de rótulos pré-definidos.

Objetivo formal: Dado $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$, encontrar uma partição $\mathcal{C} = \{C_1, \dots, C_K\}$ tal que objetos dentro de cada C_k sejam similares entre si e dissimilares aos de outros clusters.

Aplicações:

- **Segmentação de clientes:** Grupos de comportamento de compra para marketing direcionado.
- **Detecção de anomalias:** Pontos que não pertencem a nenhum cluster são candidatos a outliers.
- **Sumarização de dados:** Representa o dataset pelos centróides dos clusters (compressão).
- **Modelagem de tópicos:** Agrupa documentos por tema (text mining).
- **Bioinformática:** Agrupamento de genes por padrão de expressão.
- **Compressão de imagens:** K-Means para quantização de cores.

i Avaliação sem Ground Truth

A falta de rótulos torna a avaliação de clustering mais desafiadora do que a classificação supervisionada. Métricas internas (Silhouette, Davies-Bouldin) medem a coesão e separação dos clusters, mas podem favorecer clustering simples. Sempre combine métricas numéricas com inspeção visual (PCA, t-SNE, UMAP) para validar resultados.

3.2 K-Means

O K-Means (MacQueen, 1967; Lloyd, 1982) é o algoritmo de clustering mais popular. Minimiza a **inércia** (soma dos quadrados das distâncias intra-cluster):

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

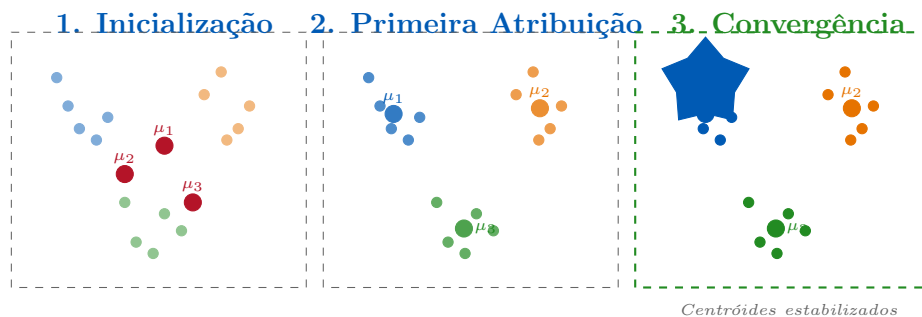
onde $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$ é o centróide do cluster k .

Algorithm 1: K-Means (Algoritmo de Lloyd)**Input:** Dados $\mathcal{X}$, número de clusters K , tolerância ε **Output:** Partição $\{C_1, \dots, C_K\}$ e centróides $\{\mu_1, \dots, \mu_K\}$

```

1 Inicializar centróides  $\mu_1, \dots, \mu_K$  aleatoriamente (ou com K-Means++);
2 repeat
    // Passo de Atribuição (E-step)
3   foreach  $\mathbf{x}_i \in \mathcal{X}$  do
4      $c_i \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \mu_k\|^2$ ;
    // Passo de Atualização (M-step)
5   foreach  $k \in \{1, \dots, K\}$  do
6      $\mu_k \leftarrow \frac{1}{|C_k|} \sum_{\mathbf{x}_i: c_i=k} \mathbf{x}_i$ ;
7 until centróides não mudam (ou  $\Delta < \varepsilon$ );

```

Visualização das etapas do K-Means:**Escolha de K:**

- **Método do Cotovelo (Elbow):** Plota inércia vs K ; o “cotovelo” indica o K ótimo.
- **Análise de Silhouette:** Para cada K , calcula o coeficiente médio; escolhe K com maior silhouette.
- **Gap Statistic** (Tibshirani et al., 2001): Compara inércia com a esperada sob distribuição de referência.

Variantes do K-Means:

- **K-Means++** (Arthur & Vassilvitskii, 2007): Inicialização melhorada – cada centróide é escolhido com probabilidade proporcional à distância ao centróide mais próximo já escolhido. Garante aproximação $O(\log K)$ do ótimo.
- **K-Medoids (PAM):** Usa exemplos reais do dataset como centróides (medóides). Mais robusto a outliers.
- **Mini-Batch K-Means:** Processa mini-lotes aleatórios; escala para datasets de milhões de exemplos.

3.3 Clustering Hierárquico

O clustering hierárquico cria uma **hierarquia** de agrupamentos representada em um **dendrograma**, sem necessidade de especificar K antecipadamente.

Agglomerativo (Bottom-up): Começa com N clusters (um por ponto) e mescla iterativamente os dois mais próximos.

Divisivo (Top-down): Começa com um único cluster e divide recursivamente.

CrITÉRIOS de Ligação (Linkage):

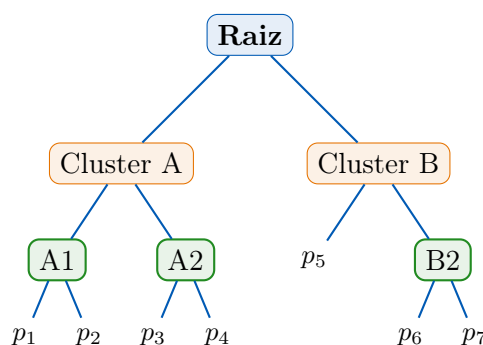
$$d_{\text{single}}(A, B) = \min_{a \in A, b \in B} d(a, b) \quad (\text{single linkage – sensível a outliers}) \quad (1)$$

$$d_{\text{complete}}(A, B) = \max_{a \in A, b \in B} d(a, b) \quad (\text{complete linkage – clusters compactos}) \quad (2)$$

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a, b) \quad (\text{UPGMA – balanço}) \quad (3)$$

$$d_{\text{ward}}(A, B) = \Delta J \text{ ao mesclar } A \text{ e } B \quad (\text{Ward – minimiza variância intra-cluster}) \quad (4)$$

Dendrograma (exemplo esquemático):



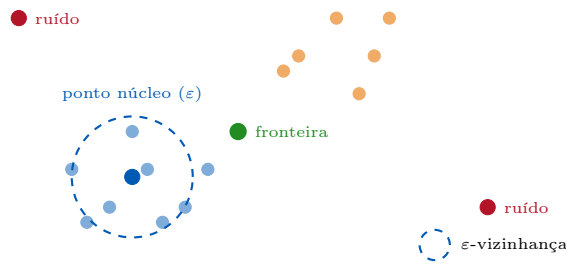
A **correlação cofenética** mede o quão fiel o dendrograma é às distâncias originais – valores > 0.75 indicam boa representação hierárquica.

3.4 DBSCAN

O **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise; Ester et al., 1996) é um algoritmo baseado em densidade que encontra clusters de **forma arbitrária** e detecta **ruído/outliers** automaticamente.

Conceitos: Dado $\varepsilon > 0$ e $\text{MinPts} \in \mathbb{N}$:

- **Ponto Núcleo:** $|N_\varepsilon(\mathbf{x})| \geq \text{MinPts}$, onde $N_\varepsilon(\mathbf{x}) = \{\mathbf{y} : d(\mathbf{x}, \mathbf{y}) \leq \varepsilon\}$.
- **Ponto de Fronteira:** Está na ε -vizinhança de um núcleo, mas não é núcleo.
- **Ponto de Ruído:** Não é núcleo nem fronteira.



DBSCAN: pontos núcleo (azul/laranja), fronteira (verde) e ruído (vermelho)

Algorithm 2: DBSCAN

Input: Dados $\mathcal{X}$, raio ε , densidade mínima MinPts

Output: Rótulos de cluster C para cada ponto (incluindo ruído = -1)

```

1  $C \leftarrow 0$ ;  marcar todos os pontos como não visitados;
2 foreach ponto não visitado  $\mathbf{p} \in \mathcal{X}$  do
3   Marcar  $\mathbf{p}$  como visitado;
4    $N \leftarrow N_\varepsilon(\mathbf{p})$  // vizinhança  $\varepsilon$ 
5   if  $|N| < \text{MinPts}$  then
6     Marcar  $\mathbf{p}$  como ruído;
7   else
8      $C \leftarrow C + 1$ ;
9     ExpandCluster( $\mathbf{p}, N, C, \varepsilon, \text{MinPts}$ );
10 Função ExpandCluster( $\mathbf{p}, N, C, \varepsilon, \text{MinPts}$ ):
11   Adicionar  $\mathbf{p}$  ao cluster  $C$ ;
12   foreach  $\mathbf{q} \in N$  do
13     if  $\mathbf{q}$  não visitado then
14       Marcar  $\mathbf{q}$  como visitado;
15        $N' \leftarrow N_\varepsilon(\mathbf{q})$ ;
16       if  $|N'| \geq \text{MinPts}$  then
17          $N \leftarrow N \cup N'$ ;
18   if  $\mathbf{q}$  ainda não atribuído a cluster then
19     Adicionar  $\mathbf{q}$  ao cluster  $C$ ;
  
```

Parâmetros: ε pode ser estimado pelo gráfico da k -NN distance (joelho da curva para $k = \text{MinPts}$). Regra prática: $\text{MinPts} \geq 2d$ onde d é a dimensionalidade.

3.5 HDBSCAN

O **HDBSCAN** (Hierarchical DBSCAN; McInnes et al., 2017) estende o DBSCAN com uma abordagem hierárquica que **varia o limiar de densidade** automaticamente.

Vantagens sobre DBSCAN:

- Detecta clusters com **densidades variáveis** (problema do DBSCAN: apenas ε global).
- Mais robusto na seleção de parâmetros: apenas `min_cluster_size` é crítico.

- Constrói uma árvore de condensação de clusters e extrai os mais persistentes.
- Retorna probabilidade de pertencimento ao cluster (métrica de membrana).

3.6 Modelos de Mistura Gaussiana (GMM)

O GMM modela os dados como uma **mistura** de K distribuições Gaussianas:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

onde π_k são os pesos da mistura ($\sum_k \pi_k = 1$, $\pi_k > 0$).

O GMM é treinado pelo **algoritmo EM** (Dempster et al., 1977):

E-step – Calcular responsabilidades:

$$r_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step – Atualizar parâmetros:

$$\pi_k = \frac{1}{N} \sum_i r_{ik}, \quad \boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{\sum_i r_{ik}}, \quad \boldsymbol{\Sigma}_k = \frac{\sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{\sum_i r_{ik}}$$

Seleção do número de componentes K : Use **BIC** (Bayesian Information Criterion) ou **AIC**: $\text{BIC} = -2 \log \hat{L} + p \log N$, onde p é o número de parâmetros. Minimize o BIC.

3.7 Spectral Clustering

O Spectral Clustering (Ng et al., 2002) baseia-se na teoria de grafos espectrais para capturar estruturas não-convexas que K-Means falha.

Algoritmo:

1. Construir matriz de similaridade $\mathbf{W}$ (ex. kernel RBF: $w_{ij} = e^{-\|x_i - x_j\|^2 / 2\sigma^2}$).
2. Calcular Laplaciano normalizado: $\mathbf{L} = \mathbf{D}^{-1/2}(\mathbf{D} - \mathbf{W})\mathbf{D}^{-1/2}$, onde $D_{ii} = \sum_j w_{ij}$.
3. Extrair os K menores autovetores de $\mathbf{L}$; formar matriz $\mathbf{U}$.
4. Aplicar K-Means no espaço dos autovetores.

3.8 Avaliação de Agrupamentos

3.8.1 Métricas Internas (sem rótulos verdadeiros)

Coeficiente de Silhouette (Rousseeuw, 1987): Para cada ponto i :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

onde $a(i)$ = distância média intra-cluster, $b(i)$ = distância média ao cluster vizinho mais próximo. $s(i) \in [-1, 1]$: valores próximos a 1 indicam boa alocação.

Índice Davies-Bouldin (Davies & Bouldin, 1979):

$$DB = \frac{1}{K} \sum_{k=1}^K \max_{k \neq k'} \frac{s_k + s_{k'}}{\|c_k - c_{k'}\|}$$

onde s_k é a distância média intra-cluster k . Minimizar DB.

Índice Calinski-Harabasz: Razão entre dispersão inter-cluster e intra-cluster. Maximizar.

3.8.2 Métricas Externas (com rótulos verdadeiros)

Adjusted Rand Index (ARI): Versão ajustada para chance do Rand Index. $ARI \in [-1, 1]$; 1 = agrupamento perfeito; 0 = aleatório.

Normalized Mutual Information (NMI):

$$NMI(U, V) = \frac{2 \cdot I(U; V)}{H(U) + H(V)} \in [0, 1]$$

V-Measure: Média harmônica de homogeneidade (cada cluster contém apenas uma classe) e completude (todos os membros de uma classe estão no mesmo cluster).

4 Regras de Associação

4.1 Conceitos Fundamentais

Dada uma base de transações $\mathcal{T} = \{T_1, \dots, T_N\}$ sobre um conjunto de itens $\mathcal{I}$, o objetivo é encontrar regras $A \Rightarrow B$ ($A, B \subseteq \mathcal{I}$, $A \cap B = \emptyset$) com alta frequência e confiabilidade.

Medidas de interesse:

$$\text{sup}(A) = \frac{|\{T \in \mathcal{T} : A \subseteq T\}|}{|\mathcal{T}|} \quad (\text{frequência do itemset}) \quad (5)$$

$$\text{sup}(A \Rightarrow B) = \text{sup}(A \cup B) \quad (\text{frequência da regra}) \quad (6)$$

$$\text{conf}(A \Rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)} \quad (\text{confiança}) \quad (7)$$

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{\text{sup}(B)} \quad (\text{importância relativa}) \quad (8)$$

$$\text{conv}(A \Rightarrow B) = \frac{1 - \text{sup}(B)}{1 - \text{conf}(A \Rightarrow B)} \quad (\text{convicção}) \quad (9)$$

Interpretação:

- **Lift** > 1: A e B são positivamente correlacionados; a regra é mais frequente que o esperado por acaso.
- **Lift** = 1: A e B são independentes.
- **Lift** < 1: A e B são negativamente correlacionados (regra negativa).

💡 Exemplo: Cesta de Mercado

Considere 5 transações: $T_1 = \{\text{leite}, \text{po}\}$, $T_2 = \{\text{leite}, \text{manteiga}, \text{po}\}$, $T_3 = \{\text{manteiga}, \text{queijo}\}$, $T_4 = \{\text{leite}, \text{manteiga}\}$, $T_5 = \{\text{leite}, \text{po}, \text{queijo}\}$.
Para a regra $\{\text{leite}\} \Rightarrow \{\text{po}\}$:

$$\text{sup}(\{\text{leite}, \text{po}\}) = 3/5 = 0.6, \quad \text{sup}(\{\text{leite}\}) = 4/5 = 0.8, \quad \text{conf} = 0.6/0.8 = 0.75$$

$$\text{sup}(\{\text{po}\}) = 3/5 = 0.6, \quad \text{lift} = 0.75/0.6 = 1.25 \quad (\text{correlação positiva})$$

4.2 Algoritmo Apriori (Agrawal & Srikant, 1994)

O Apriori usa a propriedade **anti-monótona**: se um itemset é infrequente, todos os seus superconjuntos também são. Isso permite poda da busca.

Algorithm 3: Apriori

Input: Transações $\mathcal{T}$, suporte mínimo minsup, confiança mínima minconf
Output: Conjunto de regras de associação frequentes

```

1  $L_1 \leftarrow \{\text{itemsets frequentes de tamanho } 1\}$ ;
2  $k \leftarrow 1$ ;
3 while  $L_k \neq \emptyset$  do
4    $C_{k+1} \leftarrow \text{AprioriGen}(L_k)$  // gera candidatos de tamanho  $k+1$ 
5   foreach transação  $T \in \mathcal{T}$  do
6      $D_T \leftarrow \{\text{candidatos em } C_{k+1} \text{ contidos em } T\}$ ;
7     Incrementar contagem de cada itemset em  $D_T$ ;
8    $L_{k+1} \leftarrow \{c \in C_{k+1} : \text{sup}(c) \geq \text{minsup}\}$ ;
9    $k \leftarrow k + 1$ ;
10  $\mathcal{F} \leftarrow \bigcup_k L_k$  // todos os itemsets frequentes
11 foreach  $f \in \mathcal{F}$ ,  $A \subset f$ ,  $B = f \setminus A$ ,  $A \neq \emptyset$  do
12   if  $\text{conf}(A \Rightarrow B) \geq \text{minconf}$  then
13     Adicionar  $A \Rightarrow B$  às regras;
```

4.3 Algoritmo FP-Growth (Han et al., 2000)

O FP-Growth elimina a geração de candidatos construindo uma estrutura compacta (**FP-Tree**) que representa a base de transações.

Vantagens sobre Apriori:

- Apenas **dois varrimentos** da base de dados (independente do número de itemsets).
- Sem geração explícita de candidatos.
- A FP-Tree é tipicamente muito menor que a base original (compartilha prefixos).

Funcionamento: (1) Varre a base, conta frequências de itens, ordena por frequência decrescente. (2) Varre novamente, inserindo transações na FP-Tree. (3) Extrai itemsets frequentes recursivamente usando bases de padrões condicionais.

ECLAT (Equivalence CLass Transformation): Usa formato vertical dos dados (lista de transações por item – TidList). Interseção de TidLists para computar suporte; eficiente para muitas transações.

4.4 Regras de Associação para Cenários Específicos

- **Regras Quantitativas:** Itens com atributos numéricos; discretização ou particionamento fuzzy.
- **Regras Temporais / Mineração de Sequências:** Regras com ordenação temporal; algoritmo GSP (Generalized Sequential Patterns) e PrefixSpan.
- **Regras Multi-nível:** Taxonomias de itens (ex. leite $\subset$ laticínios $\subset$ alimentos); suporte mínimo ajustado por nível.
- **Regras Negativas:** $A \Rightarrow \neg B$ (comprar A implica não comprar B); úteis para descoberta de anti-correlações.

5 Detecção de Anomalias

Tipos de Anomalias

- **Ponto:** Uma instância individual diverge significativamente das demais (ex. transação fraudulenta).
- **Contextual:** Uma instância é anômala em um contexto específico (ex. temperatura de 35°C em dezembro no RS).
- **Coletiva:** Um subconjunto de instâncias é anômalo em conjunto, mas cada instância individualmente parece normal.

Métodos de Detecção de Anomalias:

- **Estatísticos:** Assume distribuição Gaussiana; Z-score ($|z| > 3$); Grubbs Test; Boxplot ($> 1.5 \times \text{IQR}$).
- **Baseados em Distância:** K-NN Outlier Score: anomalias têm vizinhos distantes.
- **Local Outlier Factor (LOF)** (Breunig et al., 2000): Compara a densidade de um ponto com a de seus vizinhos. $\text{LOF}(p) \gg 1$ indica anomalia.
- **Isolation Forest** (Liu et al., 2008): Constrói árvores de isolamento aleatórias. Anomalias são isoladas em menos partições (caminhos curtos na árvore). Score de anomalia: $s(x, n) = 2^{-E[h(x)]/c(n)}$, onde $h(x)$ é o comprimento do caminho e $c(n)$ é o comprimento médio esperado.
- **Autoencoder:** Treina a rede neural para reconstruir dados normais. Alta **erro de reconstrução** $\|x - \hat{x}\|^2$ indica anomalia.
- **VAE (Variational Autoencoder):** Score de anomalia baseado no ELBO (Evidence Lower Bound).

6 Estado da Arte (2020–2024)

Deep Clustering

Integração de redes neurais profundas com objetivos de clustering:

- **DEC – Deep Embedded Clustering** (Xie et al., 2016): Autoencoder treinado conjuntamente com uma função de perda de clustering; aprende representações otimizadas para agrupamento.
- **SCAN** (Van Gansbeke et al., 2020): Self-supervised clustering; usa similaridade de vizinhos para treinar sem rótulos; state-of-the-art em clustering de imagens.
- **DINO** (Caron et al., 2021): Recursos do Vision Transformer (ViT) pré-treinados auto-supervisionados revelam estrutura de clustering sem fine-tuning; propriedade emergente dos Transformers.

HDBSCAN e Clustering para Dados de Alta Dimensão

- **HDBSCAN** tornou-se o padrão para clustering robusto em Python (scikit-learn ≥ 1.3). Suporta dados de alta dimensão com a métrica de alcançabilidade mútua.
- **Pipeline UMAP + HDBSCAN**: UMAP reduz dimensionalidade preservando estrutura local, HDBSCAN agrupa no espaço reduzido. Muito eficaz para NLP (embeddings de documentos), genômica single-cell (scRNA-seq).
- **BERTopic** (Grootendorst, 2022): Modela tópicos combinando BERT embeddings + UMAP + HDBSCAN + TF-IDF por cluster. Supera LDA em qualidade de tópicos.

Detecção de Outliers Moderna

- **PyOD** (Zhao et al., 2019): Biblioteca Python com 40+ algoritmos de detecção de outliers unificados em uma API sklearn-compatível.
- **COPOD** (Li et al., 2020): Copula-Based Outlier Detection; sem hiperparâmetros; muito eficiente; baseado em cópulas empíricas.
- **ECOD** (Li et al., 2022): Empirical Cumulative distribution-based Outlier Detection; computacionalmente eficiente; interpretável; sem hiperparâmetros.
- **Detecção de Anomalias em Grafos**: GraphSAGE + Isolation Forest para detecção de fraudes em redes sociais; muito relevante para Social CRM.

7 Exercícios

1. **Silhouette Manual:** Dados os pontos $p_1 = (1, 1)$, $p_2 = (2, 1)$, $p_3 = (5, 5)$, $p_4 = (6, 5)$ e os clusters $C_1 = \{p_1, p_2\}$, $C_2 = \{p_3, p_4\}$, calcule o coeficiente de silhouette para cada ponto e a silhouette média. O clustering é bom?
2. **K-Means com Diferentes K:** Aplique K-Means com $K = 2, 3, 4, 5$ ao dataset Iris (4 features). Para cada K , calcule a inércia, Silhouette e Davies-Bouldin. Plote as curvas e determine o K ótimo. O K ótimo coincide com o número real de classes (3)?
3. **Comparação de Linkages:** Aplique clustering hierárquico com single, complete, average e Ward linkage ao dataset Wine. Para cada método, corte o dendrograma em $K = 3$ e calcule ARI e NMI em relação às classes verdadeiras. Qual linkage tem melhor desempenho? Por quê?
4. **Apriori Manual:** Dada a base de transações: $T_1 = \{A, B, C\}$, $T_2 = \{A, B\}$, $T_3 = \{B, C, D\}$, $T_4 = \{A, C\}$, $T_5 = \{A, B, D\}$, $T_6 = \{B, C\}$. Com suporte mínimo = 0.5 e confiança mínima = 0.7, execute o Apriori: liste os itemsets frequentes L_1, L_2, L_3 e as regras válidas com suporte, confiança e lift.
5. **DBSCAN Parameter Selection:** Aplique DBSCAN ao dataset make_moons de scikit-learn. Varie $\epsilon \in \{0.1, 0.2, 0.3, 0.5\}$ e MinPts $\in \{3, 5, 10\}$. Para cada combinação, reporte número de clusters, número de pontos de ruído e ARI. Qual combinação é melhor? Compare com K-Means ($K = 2$).
6. **Comparação de Métodos de Anomalia:** Usando o dataset Pima Indians Diabetes ou Credit Card Fraud, compare Isolation Forest, LOF e Z-Score para detecção de anomalias. Avalie com AUC-ROC e Precisão@K ($K = 1\%$ dos dados). Qual método tem melhor desempenho?
7. **Avaliação de Clustering Pipeline:** Implemente um pipeline completo: (1) normalizar dados, (2) reduzir com PCA para 2D, (3) aplicar K-Means e HDBSCAN, (4) visualizar com t-SNE, (5) calcular todas as métricas de avaliação disponíveis. Reporte e discuta os resultados para o dataset Fashion-MNIST (10.000 amostras).

8 Referências

Referências

- [1] Agrawal, R.; Srikant, R. Fast algorithms for mining association rules. *Proceedings of the 20th VLDB Conference*, pp. 487–499, 1994.
- [2] Han, J.; Pei, J.; Yin, Y. Mining frequent patterns without candidate generation. *ACM SIGMOD Record*, 29(2):1–12, 2000.
- [3] MacQueen, J. Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1:281–297, 1967.

- [4] Arthur, D.; Vassilvitskii, S. k-means++: The advantages of careful seeding. *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.
- [5] Ester, M.; Kriegel, H.-P.; Sander, J.; Xu, X. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd ACM International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231, 1996.
- [6] McInnes, L.; Healy, J.; Astels, S. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017.
- [7] Breunig, M. M.; Kriegel, H.-P.; Ng, R. T.; Sander, J. LOF: Identifying density-based local outliers. *ACM SIGMOD Record*, 29(2):93–104, 2000.
- [8] Liu, F. T.; Ting, K. M.; Zhou, Z.-H. Isolation forest. *IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008.
- [9] Rousseeuw, P. J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- [10] Davies, D. L.; Bouldin, D. W. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2):224–227, 1979.
- [11] Dempster, A. P.; Laird, N. M.; Rubin, D. B. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B*, 39(1):1–38, 1977.
- [12] Ng, A. Y.; Jordan, M. I.; Weiss, Y. On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems*, 14, 2002.
- [13] Zhao, Y.; Nasrullah, Z.; Li, Z. PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research*, 20(96):1–7, 2019.
- [14] Caron, M.; Touvron, H.; Misra, I.; et al. Emerging properties in self-supervised vision transformers. *International Conference on Computer Vision (ICCV)*, 2021.